
Système de Billettique Numérique

Paie ment Sans Contact et QR Code Sécurisé

Valideur VPE412 — Tramway Rabat-Salé

Document de conception technique

Projet de Fin d'Études (PFE) — Simulation de déploiement futur

14 mai 2026

À propos de ce document

Ce document explique de manière simple et complète comment fonctionne le système de billettique numérique développé dans le cadre du PFE. Il couvre le processus complet : depuis le moment où le voyageur pose sa carte sur la machine jusqu'à la validation par l'agent. Des exemples concrets et des analogies du monde réel sont utilisés tout au long du document.

Table des matières

1	Comprendre le projet en 30 secondes	3
1.1	Pourquoi ce projet est utile	3
2	Les composants du système	3
2.1	Partie 1 : La machine physique (Raspberry Pi + PN532)	3
2.2	Partie 2 : Le backend (Serveur API)	4
2.3	Partie 3 : L'application web (Next.js sur Vercel)	4
3	La carte MIFARE : simulation de carte bancaire	5
3.1	Pourquoi une carte MIFARE et pas directement une vraie carte bancaire ?	5
3.2	Structure de la mémoire MIFARE Classic 1K	5
3.3	Ce qu'on écrit dans le Secteur 1	5
3.4	Sécurité de la carte en 3 niveaux	6
4	Le processus complet, étape par étape	6

4.1	Étape 0 : Enregistrement du voyageur (une seule fois)	6
4.2	Étape 1 : Le voyageur entre dans le tramway	7
4.3	Étape 2 : Lecture de la carte par le PN532	7
4.4	Étape 3 : Le backend traite la transaction	8
4.5	Étape 4 : Le QR Code apparaît sur le téléphone d’Ahmed	9
4.6	Étape 5 : L’agent scanne le QR Code avec le PDC	9
5	Base de données : structure complète	10
6	Stack technologique complet	11
7	Les cas d’usage : tous les scénarios possibles	11
7.1	Cas 1 — Voyage normal (tout fonctionne)	11
7.2	Cas 2 — QR scanné deux fois (tentative de fraude)	12
7.3	Cas 3 — QR capturé par screenshot (tentative de fraude)	12
7.4	Cas 4 — Carte MIFARE non enregistrée	12
7.5	Cas 5 — QR expiré (60 minutes dépassées)	12
7.6	Cas 6 — Perte de connexion Internet sur le RPi	12
8	Communication entre les composants	13
8.1	RPi → Serveur Vercel : comment ça se passe ?	13
8.2	Serveur → Application Web du voyageur : le temps réel	13
9	Sécurité du QR Code : pourquoi il ne peut pas être falsifié	13
9.1	Le problème : comment empêcher la fraude ?	13
9.2	La solution : HMAC-SHA256 + TOTP	14
10	Résumé visuel du cycle de vie d’un ticket	14
11	Plan de développement	15
	Conclusion	15

1 Comprendre le projet en 30 secondes

L'idée principale

Un voyageur entre dans le tramway. Il pose sa carte sur une machine (comme il paie au supermarché). 7 DH sont simulés débités. Son téléphone reçoit automatiquement un QR Code valide pendant 60 minutes. L'agent du tram scanne ce QR Code pour confirmer que le voyageur a bien payé.

Analogie réelle : C'est exactement comme le titre de transport numérique SNCF en France. Quand vous achetez un billet TGV, vous recevez un QR Code sur votre téléphone. Le contrôleur scanne ce QR Code avec son terminal. La seule différence ici : le paiement se fait sur place en tapant la carte sur la machine du tramway, et le QR arrive instantanément.

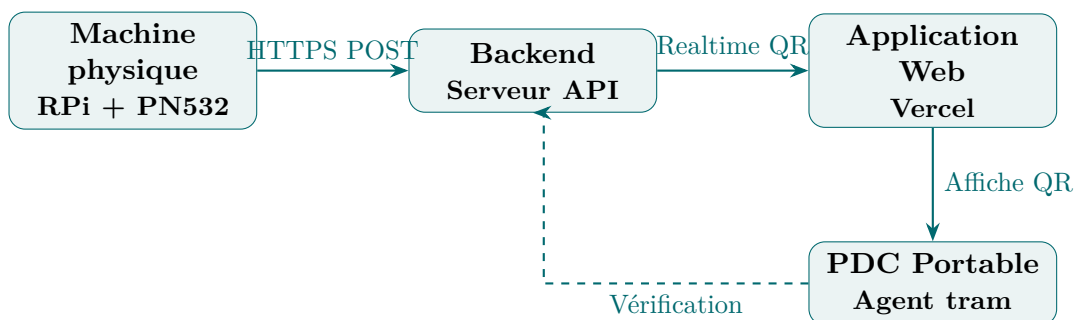
1.1 Pourquoi ce projet est utile

Aujourd'hui au Maroc, les voyageurs du tramway achètent des tickets papier ou rechargent une carte physique. Ce projet simule ce que pourrait être le tramway de demain :

- **Zéro ticket papier** : tout est numérique
- **Paiement universel** : n'importe quelle carte bancaire ou Apple Pay
- **Sécurité anti-fraude** : un QR Code ne peut pas être copié ou réutilisé
- **Expérience moderne** : similaire à ce qui existe à Dubai (RTA), Paris (RATP), Londres (TfL)

2 Les composants du système

Le système est composé de **trois parties** qui communiquent entre elles :



2.1 Partie 1 : La machine physique (Raspberry Pi + PN532)

C'est la machine présente dans le tramway. Dans notre simulation, elle est composée de :

- **Raspberry Pi 4** : le cerveau. Un mini-ordinateur qui fait tourner notre code Python et notre serveur.
- **Module PN532 NFC** : le lecteur de carte. C'est lui qui détecte quand une carte est posée dessus (comme le terminal NFC d'une caisse de supermarché).
- **Carte de simulation MIFARE Classic 1K** : simule la carte bancaire d'un voyageur.

Analogie simple

Imaginez un terminal de paiement dans un café. Vous posez votre carte bancaire dessus, il bipe, le paiement est fait. C'est exactement ce que fait notre machine. La différence : au lieu d'encaisser un commerçant, ça génère un ticket de transport numérique.

2.2 Partie 2 : Le backend (Serveur API)

C'est le serveur qui reçoit la demande de la machine, vérifie les informations, calcule le ticket, et l'envoie à l'application. Il tourne en local sur le Raspberry Pi (ou peut être déployé sur un cloud comme Railway).

Ce qu'il fait concrètement :

1. Reçoit le token de la carte (un identifiant sécurisé)
2. Cherche à quel compte voyageur il appartient
3. Simule un débit de 7 DH
4. Génère un QR Code signé et sécurisé
5. Envoie ce QR Code à l'application web du voyageur en temps réel

2.3 Partie 3 : L'application web (Next.js sur Vercel)

C'est l'application que le voyageur utilise sur son téléphone. Elle sert à :

- Créer son compte (email + mot de passe)
- Vérifier son email pour activer le compte
- Voir s'afficher automatiquement le QR Code après paiement
- Consulter l'historique de ses voyages

L'application est déployée sur **Vercel** (serveur dans le cloud) : elle est accessible depuis n'importe quel téléphone avec Internet.

3 La carte MIFARE : simulation de carte bancaire

3.1 Pourquoi une carte MIFARE et pas directement une vraie carte bancaire ?

Une vraie carte bancaire utilise le protocole EMV (Europay Mastercard Visa). Pour traiter un vrai paiement bancaire, il faut un terminal certifié PCI-DSS (coûte plusieurs milliers d'euros) et un contrat avec une banque. Dans le cadre d'un PFE de simulation, on utilise une carte MIFARE qui imite le comportement d'une carte sans modifier les vraies banques.

Analogie réelle

C'est comme les simulateurs de vol pour les pilotes. L'Airbus A320 de simulation n'est pas un vrai avion, mais il reproduit exactement le comportement réel. Notre carte MIFARE reproduit le comportement d'une carte bancaire dans notre contexte de simulation.

3.2 Structure de la mémoire MIFARE Classic 1K

La carte MIFARE Classic 1K possède 1024 octets de mémoire, organisés en 16 secteurs :

Secteur	Blocs	Taille	Utilisation dans notre projet
0	0-3	64 oct.	Réservé fabricant. Bloc 0 = UID unique (lecture seule)
1	4-7	64 oct.	Nos données : TOKEN sécurisé + version carte
2	8-11	64 oct.	Métadonnées : date expiry + statut (ACTIVE/BLOCKED)
3-15	12-63	832 oct.	Non utilisés (réservés pour futures fonctionnalités)

3.3 Ce qu'on écrit dans le Secteur 1

On n'écrit **jamais** de données bancaires réelles dans la carte. On écrit un token sécurisé :

Exemple de ce qui est écrit dans le Bloc 4 (Secteur 1)

```

1 # Ce qu'on voit dans le bloc 4 de la carte
2 bloc_4 = "A3F9B2C1D4E5F601" # 16 octets = 16 caracteres
3
4 # Ce token est genere comme ceci :
5 import hashlib, os
6
7 user_id = "42" # l'ID du voyageur dans notre
  base
8 salt = "7f3a9c2b" # aleatoire, unique par carte
9 secret = "SECRET_APP_KEY" # connu uniquement du serveur

```

```

10
11 token = hashlib.sha256(
12     f"{user_id}{salt}{secret}".encode()
13 ).hexdigest()[:16].upper()
14
15 # Resultat : "A3F9B2C1D4E5F601"
16 # Sans le salt et le secret -> impossible de retrouver user_id
    =42

```

3.4 Sécurité de la carte en 3 niveaux

1. **Clé d'accès secteur** : le Secteur 1 est protégé par une Clé A de 6 octets. Sans cette clé, impossible de lire le contenu. Seul notre Raspberry Pi connaît cette clé.
2. **Donnée hashée** : même si quelqu'un arrive à lire le bloc, il obtient un token comme A3F9B2C1D4E5F601. Celui-ci ne signifie rien sans le sel et la clé secrète du serveur.
3. **Double vérification en base** : le token sur la carte doit correspondre à une entrée active dans Supabase. Un token modifié manuellement ne sera jamais trouvé.

Important : en production réelle

Dans un vrai déploiement, la machine utiliserait un vrai terminal EMV certifié (ex : PAX, Ingenico) connecté à un prestataire de paiement (Stripe, CMI Maroc). Notre simulation avec MIFARE reproduit la logique mais pas les protocoles bancaires réels.

4 Le processus complet, étape par étape

4.1 Étape 0 : Enregistrement du voyageur (une seule fois)

Avant d'utiliser le système, le voyageur doit créer son compte et lier sa carte MIFARE :

1. Le voyageur ouvre l'application web sur son téléphone (`tram-app.vercel.app`)
2. Il clique sur « **Créer un compte** » et saisit son email et mot de passe
3. Supabase envoie un email avec un lien de confirmation
4. Il clique sur le lien → compte activé ✓
5. Il se rend à une borne d'enregistrement (ou l'opérateur utilise le RPi en mode admin)
6. Il pose sa carte MIFARE sur le lecteur PN532
7. Le RPi génère un token unique, l'écrit dans le Secteur 1 de la carte

8. Ce token est sauvegardé dans la base de données lié à son compte
9. La carte est prête à être utilisée ✓

Exemple concret

Ahmed crée un compte avec `ahmed@gmail.com`. Son `user_id` dans la base est 42. Le système génère le token `A3F9B2C1D4E5F601` et l'écrit dans sa carte. Dans la base de données, la ligne `cards(token="A3F9B2C1...", user_id=42, status="ACTIVE")` est créée. Désormais, quand Ahmed tape sa carte, le système sait que c'est Ahmed.

4.2 Étape 1 : Le voyageur entre dans le tramway

Ahmed entre dans le tramway. Il voit la machine (notre Raspberry Pi + PN532). Il pose sa carte MIFARE dessus (comme il poserait sa carte bancaire sur un terminal de paiement).

4.3 Étape 2 : Lecture de la carte par le PN532

Le module PN532 détecte la carte en moins d'une seconde :

```
Script Python - nfc_reader.py (simplifié)

1 import board, busio, requests
2 from adafruit_pn532.i2c import PN532_I2C
3
4 # Initialisation PN532 en mode I2C
5 i2c = busio.I2C(board.SCL, board.SDA)
6 pn532 = PN532_I2C(i2c, debug=False)
7 pn532.SAM_configuration()
8
9 SECRET_KEY_A = [0xA0, 0xB1, 0xC2, 0xD3, 0xE4, 0xF5] # cle
    secrete
10
11 print("En attente d'une carte...")
12
13 while True:
14     uid = pn532.read_passive_target(timeout=0.5)
15     if uid is None:
16         continue
17
18     print(f"Carte detectee ! UID = {[hex(i) for i in uid]}")
19
20     # Authentifier et lire le bloc 4 (secteur 1)
21     pn532.mifare_classic_authenticate_block(
22         uid, block_number=4,
23         key_number=0,          # Cle A
24         key=SECRET_KEY_A
25     )
26     bloc_data = pn532.mifare_classic_read_block(4)
27     token = bytes(bloc_data[:16]).decode('ascii').strip()
28
```

```

29     # Envoyer au backend local
30     response = requests.post("http://localhost:3000/api/tap",
31                               json={
32                                   "token": token,
33                                   "amount": 7,
34                                   "secret": "Rpi_MACHINE_SECRET"
35                               })
36
37     print(f"Reponse backend : {response.json()}")
38     break

```

4.4 Étape 3 : Le backend traite la transaction

Le serveur Express reçoit le token. Il fait les vérifications suivantes dans cet ordre :

1. Est-ce que la requête vient bien d'une machine autorisée? (vérification du RPi_MACHINE_SECRET)
2. Ce token existe-t-il dans la base de données?
3. Ce compte est-il actif (pas bloqué)?
4. Simuler le débit de 7 DH ✓
5. Générer le QR Code sécurisé
6. Envoyer le QR en temps réel à l'application web du voyageur

Génération du QR Code sécurisé (Node.js)

```

1 // Pseudo-code Node.js
2 const crypto = require('crypto');
3
4 function generateSecureTicket(userId, transactionId) {
5     const timestamp = Math.floor(Date.now() / 1000);
6     const window15s = Math.floor(timestamp / 15); // fenetre 15
7     const expiresAt = timestamp + 3600;           // +60
8     const payload = {
9         uid:      userId,
10        txId:     transactionId,
11        win:      window15s,
12        exp:      expiresAt,
13        ts:       timestamp
14    };
15
16    // Signer le payload avec HMAC-SHA256
17    const signature = crypto
18        .createHmac('sha256', process.env.HMAC_SECRET)
19        .update(JSON.stringify(payload))
20        .digest('hex');
21

```



```

22
23   payload.sig = signature;
24
25   // Le QR Code contient ce payload encode en Base64
26   return Buffer.from(JSON.stringify(payload)).toString('
      base64url');
27 }
28 // Exemple de QR Code genere :
29 // "eyJ1aWQiOiJyLCJ0eElkIjo1VFgwMDEiLCJ3aW4i..."

```

4.5 Étape 4 : Le QR Code apparaît sur le téléphone d’Ahmed

L’application web reçoit le QR Code via Supabase Realtime (WebSocket). La page se rafraîchit automatiquement et affiche le QR Code. Ahmed n’a rien à faire, il n’a même pas besoin d’ouvrir l’application manuellement.

Le QR Code affiché contient :

- Un identifiant de transaction unique
- L’heure de génération + l’heure d’expiration (60 minutes)
- Une signature HMAC-SHA256 qui garantit son authenticité
- Une fenêtre temporelle de 15 secondes (anti-capture d’écran)

Exemple affiché sur le téléphone d’Ahmed

[QR CODE IMAGE]

Tramway Rabat-Salé

Titre de transport valide

Valable 60 min à partir de 13h30

Expire à 14h30

4.6 Étape 5 : L’agent scanne le QR Code avec le PDC

L’agent du tramway utilise le PDC (Portable de Contrôle) ACS. Il scanne le QR Code sur l’écran du téléphone d’Ahmed. Le PDC envoie le contenu du QR au serveur qui effectue 3 vérifications :

Vérification	Résultat possible	Explication
Signature HMAC	Valide / Invalide	Le QR a-t-il été falsifié ?
Expiration	Valide / Expiré	Est-il dans les 60 minutes ?
Unicité	Non utilisé / Déjà utilisé	A-t-il déjà été scanné ?

QR VALIDE

Les 3 vérifications passent.
 Affichage vert :
Titre de transport valide
 Valable jusqu'à 14h30
 PDC001 — Agent 12
 10 :32 :15 — 20/05/2025

QR INVALIDE

Une vérification échoue.
 Affichage rouge :
Titre invalide ou expiré
 Déjà utilisé / Expiré
 PDC001 — Agent 12
 10 :32 :15 — 20/05/2025

5 Base de données : structure complète

La base de données Supabase contient 4 tables principales :

Schéma SQL - Supabase (PostgreSQL)

```
-- Table 1 : Comptes voyageurs
-- Creee automatiquement par Supabase Auth
-- auth.users(id, email, created_at, email_confirmed_at)

-- Table 2 : Profils voyageurs (etend auth.users)
CREATE TABLE profiles (
  id          UUID PRIMARY KEY REFERENCES auth.users(id),
  full_name   TEXT,
  phone       TEXT,
  created_at  TIMESTAMP DEFAULT now()
);

-- Table 3 : Cartes MIFARE enregistrees
CREATE TABLE cards (
  id          SERIAL PRIMARY KEY,
  user_id     UUID REFERENCES auth.users(id),
  card_token  TEXT UNIQUE NOT NULL, -- ex: "A3F9B2C1D4E5F601"
  label       TEXT DEFAULT 'Ma carte',
  status      TEXT DEFAULT 'ACTIVE', -- ACTIVE, BLOCKED
  created_at  TIMESTAMP DEFAULT now()
);

-- Table 4 : Tickets generes (historique des voyages)
CREATE TABLE tickets (
  id          SERIAL PRIMARY KEY,
  user_id     UUID REFERENCES auth.users(id),
  card_id     INTEGER REFERENCES cards(id),
  transaction_id TEXT UNIQUE NOT NULL, -- ex: "TX-2025-001234"
```

```

qr_payload      TEXT NOT NULL,          -- le QR Code encode
amount          DECIMAL DEFAULT 7.00,
status          TEXT DEFAULT 'PENDING', -- PENDING, USED,
              EXPIRED
generated_at     TIMESTAMP DEFAULT now(),
expires_at       TIMESTAMP,
used_at          TIMESTAMP
);

```

Exemple de données réelles en base :

Table cards	card_token	user_id	status
1	A3F9B2C1D4E5F601	42	ACTIVE
2	B7E2A4D9C1F30812	87	ACTIVE
3	CC1A3F7B9D2E4501	15	BLOCKED

6 Stack technologique complet

Couche	Technologie	Rôle
Matériel	Raspberry Pi 4	Cerveau de la machine tram
Lecture carte	PN532 (I2C)	Lire le token de la carte MIFARE
Simulation carte	MIFARE Classic 1K	Simule la carte bancaire voyageur
Script lecture	Python 3 + adafruit-pn532	Lire et écrire sur la carte
Serveur local	Node.js + Express	API locale sur le Raspberry Pi
Auth & DB	Supabase	Comptes, cartes, tickets, realtime
Application web	Next.js 14 + React	Interface voyageur sur téléphone
Déploiement web	Vercel	Héberge l'application (HTTPS auto)
Sécurité QR	HMAC-SHA256 + TOTP	Anti-fraude, anti-capture d'écran
PDC (contrôle)	App ACS PDC Contrôle	Scan QR par l'agent

7 Les cas d'usage : tous les scénarios possibles

7.1 Cas 1 — Voyage normal (tout fonctionne)

1. Ahmed tape sa carte sur la machine
2. Token lu → compte trouvé → 7 DH simulés → QR généré
3. QR s'affiche sur le téléphone d'Ahmed en moins de 3 secondes
4. Agent scanne → **QR VALIDE** ✓

7.2 Cas 2 — QR scanné deux fois (tentative de fraude)

1. Ahmed montre son QR → Agent 1 scanne → **VALIDE** ✓ → ticket marqué **USED** en base
2. Ahmed essaie de le montrer à un deuxième agent
3. Agent 2 scanne → backend vérifie → status = **USED**
4. Agent 2 voit **QR INVALIDE** ✗ — **Déjà utilisé**

7.3 Cas 3 — QR capturé par screenshot (tentative de fraude)

1. Ahmed fait une capture d'écran de son QR Code
2. Le QR change automatiquement toutes les 15 secondes
3. Le screenshot contient un ancien QR qui n'est plus valide
4. Agent scanne le screenshot → **QR INVALIDE** ✗ — **Expiré**

7.4 Cas 4 — Carte MIFARE non enregistrée

1. Un voyageur pose une carte qui n'est liée à aucun compte
2. Le backend reçoit un token inconnu
3. La machine affiche : « Carte non reconnue. Veuillez créer un compte. »
4. Le voyageur est redirigé vers l'application web pour créer son compte

7.5 Cas 5 — QR expiré (60 minutes dépassées)

1. Ahmed a payé à 10h30 mais monte dans le tramway à 11h45
2. Son QR a expiré à 11h30 (60 min après génération)
3. Agent scanne → **QR INVALIDE** ✗ — **Expiré**
4. Ahmed doit retaper sa carte pour générer un nouveau QR

7.6 Cas 6 — Perte de connexion Internet sur le RPi

1. La connexion Internet du Raspberry Pi est coupée
2. Le mode dégradé s'active automatiquement
3. Les transactions sont stockées localement (SQLite sur le RPi)
4. Quand Internet revient, tout est synchronisé vers Supabase
5. Les QR générés en mode dégradé sont valides (clé pré-partagée locale)

8 Communication entre les composants

8.1 RPi → Serveur Vercel : comment ça se passe ?

Le Raspberry Pi envoie une requête HTTPS vers l'API Next.js déployée sur Vercel. C'est comme quand vous faites une recherche sur Google : votre téléphone envoie une requête à un serveur dans le cloud.

```
Exemple de requête du RPi vers Vercel

1 POST https://tram-app.vercel.app/api/tap
2 Content-Type: application/json
3 Authorization: Bearer RPi_MACHINE_SECRET_TOKEN
4
5 {
6   "token": "A3F9B2C1D4E5F601",
7   "amount": 7,
8   "machine_id": "MACHINE_TRAM_L1_ARRET_03"
9 }
10
11 # Reponse du serveur (en moins de 500ms) :
12 {
13   "success": true,
14   "ticket_id": "TX-2025-001234",
15   "user_id": 42,
16   "expires_at": "2025-05-14T14:30:00Z",
17   "message": "Ticket genere avec succes"
18 }
```

8.2 Serveur → Application Web du voyageur : le temps réel

Dès que le serveur génère le ticket, il doit l'envoyer immédiatement sur le téléphone du voyageur sans qu'Ahmed ait à rafraîchir la page. C'est ce qu'on appelle le **temps réel**. On utilise Supabase Realtime pour cela.

Analogie simple

C'est comme les notifications WhatsApp. Quand quelqu'un vous envoie un message, votre téléphone le reçoit immédiatement sans que vous ayez besoin d'ouvrir l'application et d'appuyer sur « Rafraîchir ». Supabase Realtime fonctionne de la même manière : la page du voyageur est connectée en permanence et reçoit le QR Code dès qu'il est généré.

9 Sécurité du QR Code : pourquoi il ne peut pas être falsifié

9.1 Le problème : comment empêcher la fraude ?

Sans sécurité, n'importe qui pourrait :

— Faire une capture d'écran du QR Code d'un ami et l'utiliser

- Générer un faux QR Code avec une application QR generator
- Utiliser le même QR Code plusieurs fois

9.2 La solution : HMAC-SHA256 + TOTP

Le QR Code contient une **signature numérique** (comme une empreinte digitale). Cette signature est calculée avec une clé secrète connue uniquement du serveur. Voici pourquoi cela fonctionne :

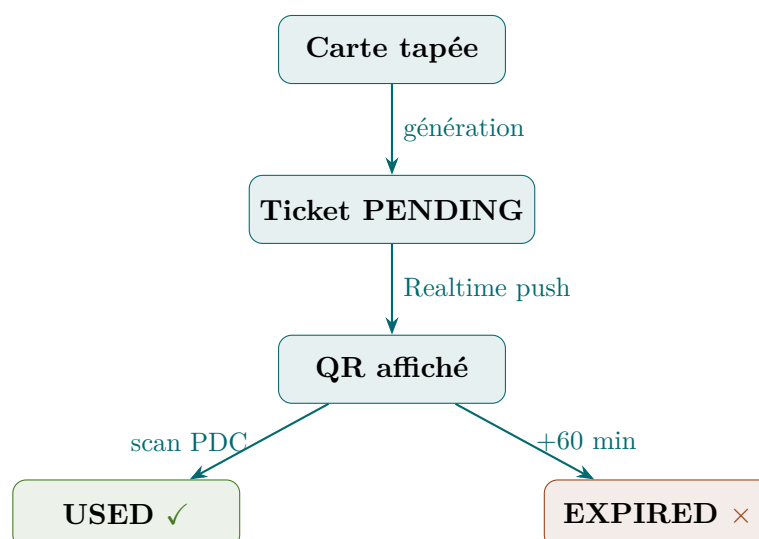
$$\text{signature} = \text{HMAC-SHA256}(\text{clé secrète}, \text{payload})$$

Si quelqu'un modifie une seule lettre du QR Code, la signature ne correspond plus → QR Code invalide.

De plus, le QR Code contient une fenêtre temporelle de 15 secondes (inspiré du standard TOTP utilisé dans les applications 2FA comme Google Authenticator). Un QR Code généré à 13:30:00 est invalide à 13:30:16.

Mécanisme	Protège contre	Comment
HMAC-SHA256	Falsification	Signature vérifiable côté serveur
Fenêtre 15s	Capture d'écran	QR change toutes les 15 secondes
Expiry 60min	Réutilisation tardive	Ticket valide 1h seulement
Unicité txId	Double scan	Ticket marqué USED après 1er scan
Clé machine	Fausse machines	Seuls les RPi autorisés peuvent taper

10 Résumé visuel du cycle de vie d'un ticket



11 Plan de développement

Sprint	Objectif	Livrables
S1	Hardware & Lecture NFC	PN532 câblé, script Python lit MIFARE, token extrait
S2	Backend & Base de données	API Express/Node.js, Supabase configuré, route <code>/api/tap</code>
S3	Application Web & QR	Next.js + Auth Supabase, QR affiché en temps réel
S4	Tests & Rapport PFE	Démonstration complète, rapport LaTeX, soutenance

Conclusion

Ce projet simule un système de billettique numérique moderne pour le tramway de Rabat-Salé. L'architecture repose sur trois composants qui communiquent : la machine physique (Raspberry Pi + PN532) qui lit les cartes, le serveur backend qui traite les paiements et génère des tickets sécurisés, et l'application web que le voyageur utilise pour afficher son QR Code.

La sécurité est assurée à plusieurs niveaux : la carte MIFARE ne contient qu'un token hashé (jamais de données sensibles réelles), les QR Codes sont signés avec HMAC-SHA256 et changent toutes les 15 secondes, et chaque ticket ne peut être utilisé qu'une seule fois.

Ce système peut évoluer vers un déploiement réel en remplaçant la carte MIFARE par un vrai terminal EMV certifié et en intégrant un prestataire de paiement bancaire agréé au Maroc (CMI).